

Detection of Traffic Violations and Congestion Using Deep Learning

Chinta Gkentian¹ and Kostas Kolomvatsos¹

¹Department of Informatics and Telecommunications, University of Thessaly, 3rd Km of the Old National Road Lamia - Athens, 35100, Lamia, Greece

¹gchinta@uth.gr, kostasks@uth.gr

Abstract

Nowadays, one can observe an increase in traffic violations, which potentially can lead to the loss of lives. A violation occurs when a driver violates traffic law with a vehicle in motion. Additionally, traffic congestion affects citizens' lives not only because they spend much time in idle vehicles but also because it increases environmental pollution and fuel consumption. It becomes obvious that if we rely on automated systems that can recognize both 'events,' we can easily apply mitigation actions and reduce the loss of lives and greenhouse emissions while simultaneously increasing the quality of living for citizens. In this paper, we develop a real-time traffic violation and congestion detection system using deep learning methodologies. Our system leverages the YOLOv5 (You Only Look Once version 5) algorithm with an EfficientNet B3 backbone for enhanced detection accuracy and tracking of vehicles, including the identification of stop sign violations. Additionally, the system has the capability to recognize stop signs, and upon detecting a stop sign, it monitors the speed of cars, issuing alerts if a car does not come to a complete stop. A sorting algorithm is integrated for vehicle tracking, providing video and text-based output with vehicle positions. The system also monitors traffic flow, alerting users when congestion exceeds a predefined threshold. While effective, limitations exist in speed and distance estimation due to factors such as camera placement and hardware constraints. Future enhancements may focus on improving these areas and expanding vehicle type detection for better traffic safety and compliance monitoring.

1 Introduction

Modern urban traffic management faces numerous challenges due to the rise in urbanization and the growing number of vehicles on the roads¹. Congestion is exacerbated by the proliferation of ride-sharing services, delivery vehicles, and personal transportation, turning daily commutes into prolonged and stressful experiences. This issue is particularly severe in metropolitan areas, where high vehicle density, unpredictable traffic patterns, and inadequate infrastructure result in frequent bottlenecks. Studies show that traffic congestion in major global cities leads to billions of dollars in lost productivity annually, significantly increasing commuters' travel time. Additionally, the environmental impact, such as heightened air pollution and greenhouse gas emissions, further complicates urban planning and sustainability efforts. Efficient traffic management and adherence to traffic regulations are essential for ensuring road safety and maintaining smooth vehicular flow [3]. Compliance with regulations such as speed limits, stop signs, and traffic signals plays a pivotal role in reducing accidents and protecting vulnerable road users, including pedestrians and cyclists. However, traditional enforcement methods, which depend on manual oversight, are often resource-intensive, prone to delays, and susceptible to human error. These approaches also struggle to provide real-time monitoring and enforcement, particularly under dynamic traffic conditions, thereby limiting their effectiveness. Violations such as illegal parking, reckless driving, and failure to obey traffic signals further exacerbate congestion, especially during peak hours. The lack of real-time, effective enforcement has led to an increase in such violations, particularly in densely populated urban areas. This underscores the need for advanced, automated systems that can monitor traffic in real-time and enforce compliance with traffic regulations.

In response to existing challenges in traffic monitoring, this paper introduces several key enhancements and functionalities:

- **Integration of EfficientNet B3:** The system integrates the YOLOv5 object detection algorithm with the EfficientNet B3 backbone [1], enhancing detection accuracy by optimizing the balance between network depth, width, and resolution. This modification improves the system's ability to process high-resolution visual data without sacrificing computational efficiency.
- **Use of the SORT Algorithm:** By incorporating the SORT algorithm², the system achieves precise tracking of vehicle movements across multiple frames. This allows for a detailed analysis of traffic patterns and behaviors, enhancing the detection of anomalies and regular traffic flow.
- **Real-Time Traffic Monitoring and Violation Detection:** The system excels in real-time monitoring, capable of:
 - Detecting and recording vehicle positions.
 - Identifying stop signs and stop sign violations.
 - Estimating vehicle speeds in real-time, thereby enabling proactive enforcement of speed-related traffic regulations.
 - Monitoring traffic congestion and issuing alerts when vehicle density exceeds predefined thresholds.

The system not only provides visual outputs but also generates detailed data logs for further analysis. This multifaceted approach significantly improves road safety, reduces congestion, and enhances compliance with traffic laws through effective real-time monitoring and post-event analysis. These improvements and functionalities make the proposed system superior to conventional traffic monitoring solutions, offering a more robust and adaptable framework for enhancing urban traffic safety and efficiency.

The paper is organized as follows. Section 2 reports on the related work while Section 3 discusses the adopted deep learning algorithms and the proposed architecture. Section 4 discusses a use case on example videos and Section 6 conclude our paper by presenting the future directions.

¹<https://www.tutorchase.com/answers/ib/geography/how-does-urbanisation-impact-traffic-congestion-trends>

²<https://www.researchgate.net/figure/The-Architecture-of-YoloV5f-ig5358622938>

2 Related Work

Numerous approaches have been employed to improve traffic violation detection using deep learning models. In their 2022 study, Kumar et al. [4] investigate the application of deep learning for object detection in video frames, with a focus on traffic management, surveillance, and self-driving systems. The authors compare traditional object detection techniques with modern deep learning models, such as YOLO and R-CNN, demonstrating the superiority of deep neural networks for real-time detection and feature extraction. They also address key challenges such as occlusion, lighting variations, and object scaling, which are critical in real-time video analysis. While our system also leverages YOLOv5 for real-time vehicle detection, we enhance detection accuracy by integrating the EfficientNet B3 backbone, which improves performance under similar conditions of occlusion and lighting variability. Pavlov et al. [7] present a comprehensive video analysis system based on machine learning and computer vision algorithms, incorporating stages such as face detection, tracking, and classification tasks. Although their study centers on human-centric analysis, the use of multiple machine learning techniques to enhance classification accuracy parallels our approach.

Significant advancements in vehicle detection using deep learning have been made, such as the 2022 project at Shandong University [12], which employed the YOLO algorithm for high-precision object detection in video streams. Their work demonstrates YOLO’s capabilities for vehicle detection with high accuracy, similar to our project. However, our system improves on this by combining YOLOv5 with EfficientNet B3, which enhances computational efficiency and allows for higher-resolution input processing in real-time traffic monitoring. Similarly, Priyadarshan’s research at The University of Texas at Austin [6] applied deep learning to comprehensive traffic management, focusing on the prediction and integration of autonomous vehicles. While their work emphasizes improving traffic flow and vehicle coordination, our system focuses on traffic rule compliance, detecting violations such as stop sign infractions and estimating vehicle speeds. This capability facilitates direct enforcement of traffic laws in addition to improving overall traffic flow.

Further advancements in traffic violation detection are discussed in a 2020 study from India [?], where YOLO and CNNs were combined to detect helmet and crosswalk violations. Their system demonstrated high precision under challenging conditions, such as occlusion and variable illumination, which are common issues in traffic monitoring. Our system expands on this by incorporating the sorting tracking algorithm to maintain vehicle identification across video frames, thus improving the system’s reliability in dynamic environments. Additionally, Nugroho et al. [5] explored the integration of YOLOv5 with MobileNetV3 for real-time detection tasks beyond traffic management, demonstrating the broader applicability of deep learning models. While their study focuses on other domains, it shares methodological similarities with our work, particularly in using lightweight models for real-time detection. However, our system is specifically tailored for traffic violation detection and real-time traffic monitoring.

Collectively, these studies demonstrate the growing potential of deep learning technologies in revolutionizing traffic management, video processing, and safety systems. Building on these advancements, our system introduces a novel combination of YOLOv5, EfficientNet B3, a multi-scale feature fusion technique, and a sorting algorithm to enhance real-time traffic violation detection, offering both high detection accuracy and efficient tracking capabilities. This combination aims to improve road safety, congestion management, and the enforcement of traffic regulations.

3 The Deep Learning Model Architecture

3.1 You only look once (YOLO)

Object detection in computer vision focuses on locating and classifying objects within images or video frames. This process involves identifying the precise locations of objects, which is crucial for applications such as autonomous driving, surveillance, and augmented reality. Convolutional neural networks (CNNs) ³, including models like Faster R-CNN ⁴, YOLO (Yolo Look Only Once) ⁵, and SSD (Single

³<https://blog.roboflow.com/what-is-a-convolutional-neural-network/>

⁴<https://blog.roboflow.com/what-is-r-cnn/>

⁵<https://docs.ultralytics.com>

Shot MultiBox Detector)⁶, are widely used for real-time object detection. The object detection process typically involves using pre-trained CNN models to extract features from images. These features help predict class labels and bounding boxes for each detected object. Post-processing techniques, such as non-maximum suppression, are used to refine results and eliminate duplicate detections. YOLO is a prominent object detection technique that predicts bounding boxes and class probabilities simultaneously for each grid cell in an image. We chose YOLOv5 for its speed and accuracy, essential for real-time traffic violation detection. Its single-pass processing makes it highly efficient, while its ability to handle objects of various sizes in complex scenes. YOLO divides the image into an $S \times S$ grid (e.g., $S = 7$ for YOLOv1), with each cell responsible for detecting objects whose centers fall within it. The system predicts multiple bounding boxes and associated confidence scores for each cell. Confidence scores reflect the Intersection over Union (IOU) between predicted and ground truth boxes. Additionally, YOLO predicts conditional class probabilities, indicating the likelihood of each class given an object's presence. During inference, these probabilities are combined with confidence scores to determine the final detection results.

In YOLO, the backbone network, which handles feature extraction, significantly impacts the system's efficiency and accuracy. The backbone processes the image to create feature maps, which are then refined through the neck of the network and used by the head to make final predictions. The head can be implemented using different approaches, such as parameter-coupled or parameter-decoupled methods, or anchor-based and anchor-free techniques. Changing the backbone to EfficientNet B3 is essential for improving feature extraction efficiency and enhancing detection accuracy in the YOLO architecture. The following equations stand true.

$$\text{Loss}_{obj} = \text{Loss}_{class} + \text{Loss}_{box} + \text{Loss}_{conf} \quad (1)$$

$$L_{class} = - \sum_{i=0}^{S*S} I_{ij}^{obj} \sum_{c \in \text{classes}} \left[\hat{P}_i^j \log P_i^j + (1 - \hat{P}_i^j) \log(1 - \hat{P}_i^j) \right] \quad (2)$$

$$L_{box} = 1 - \text{CIoU} \quad (3)$$

$$L_{conf} = - \sum_{i=0}^{S*S} I_{ij}^{obj} \left[\hat{M}_i^j \log M_i^j + (1 - \hat{M}_i^j) \log(1 - M_i^j) \right] \quad (4)$$

$$- \lambda_{noobj} \sum_{i=0}^{S*S} \sum_{j=0}^N I_{ij}^{noobj} \left[\hat{M}_i^j \log M_i^j + (1 - \hat{M}_i^j) \log(1 - M_i^j) \right]$$

Notable changes in YOLOv5 [11] compared to previous versions include replacing the Focus structure with a Conv2d 6 x 6 structure for improved efficiency and substituting SPP (Spatial Pyramid Pooling) with SPPF (Spatial Pyramid Pooling - Fast) in the neck, which more than doubles processing speed. For data augmentation, YOLOv5 incorporates various techniques to enhance model robustness. Mosaic Augmentation combines four training images into one, improving the model's ability to handle different scales and object translations. Copy-Paste Augmentation involves copying segments from one image and pasting them onto another, creating new training samples. Random Affine Transformations apply random rotations, scaling, translations, and cropping to images. MixUP Enhancement creates composite images by linearly combining two images and their labels. YOLOv5 also utilizes Albumentations, an image augmentation library, and HSV Enhancement to introduce random changes in color, saturation, and value. Random Horizontal Flip is used to flip images horizontally at random. For training, YOLOv5 employs strategies such as Multiscale Training, AutoAnchor, Warmup and Cosine LR Scheduler, Exponential Moving Average (EMA), Mixed Precision Training, and Hyperparameter Evolution to improve model performance.

⁶<https://towardsdatascience.com/understanding-ssd-multibox-real-time-object-detection-in-deep-learning-495ef744fab>

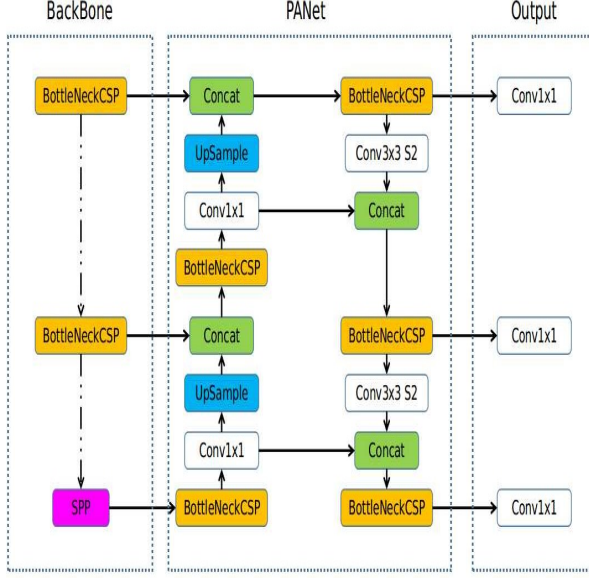


Figure 1: The architecture of YOLOv5^a

^a<https://www.researchgate.net/figure/The-Architecture-of-YoloV5fig5358622938>

The architecture of YOLOv5 consists of three main components:

1. **Backbone:** It uses CSP-Darknet53, previously used in YOLOv4. This component serves as the foundational network for feature extraction.
2. **Neck:** This component acts as a link between the backbone and the head. In YOLOv5, it includes SPPF (Spatial Pyramid Pooling Fusion) and the new CSP-PAN, which enhance processing speed.
3. **Head:** It utilizes the YOLOv3 head to produce the final detection results.

3.2 Application of SORT Algorithm in Traffic Violation Detection

The SORT (Simple Online and Realtime Tracking)⁷ algorithm is used in our system to maintain consistent identities for vehicles across consecutive video frames. SORT is a lightweight multi-object tracking framework specifically designed for real-time applications, and it is not related to classical sorting algorithms. Instead, it combines Kalman filtering and the Hungarian assignment algorithm to associate detections with existing tracked objects.

Once YOLOv5 provides the bounding boxes for each frame, SORT performs two main steps:

1. **Motion Prediction:** Each active track is modeled with a linear Kalman filter, which predicts the object's bounding box position in the next frame. This prediction allows the tracker to remain stable even under temporary detection loss, partial occlusion, or abrupt motion changes.

2. **Data Association:** After predictions are made, SORT pairs the predicted tracks with the new YOLOv5 detections. An IoU-based cost matrix is constructed, and the Hungarian algorithm is used to optimally assign detections to tracks. This minimizes identity switches and ensures consistent tracking over time.

Tracks that fail to receive detections for several consecutive frames are terminated, while unmatched detections initialize new tracks. Through this predictive-associative cycle, SORT maintains accurate and robust vehicle trajectories throughout the video.

This reliable tracking mechanism is essential for the downstream components of our system, including speed estimation, stop-sign violation detection, and real-time congestion monitoring. The integration of SORT with YOLOv5 enables efficient, real-time analysis of dynamic traffic scenes.

3.3 YOLO Backbone Update

In the original YOLOv5 architecture, the backbone used is CSPDarknet53 [?], a custom CNN designed to balance speed and accuracy. This balance is achieved through Cross Stage Partial (CSP) connections and Darknet layers, which are efficient in extracting features critical for detecting objects across various scales. By integrating EfficientNet B3 [1] as the new backbone, we replace CSPDark-

⁷<https://github.com/abewley/sort>

net53 with a network renowned for its superior performance and computational efficiency. EfficientNet B3 employs depthwise separable convolutions and squeeze-and-excitation blocks, along with a carefully balanced network depth, width, and resolution. These features reduce computational load while maintaining high-quality feature extraction. The integration of EfficientNet B3 requires adjustments to the interfacing layers of YOLOv5 to ensure seamless integration with the existing neck and head components of the model. These adjustments are crucial for maintaining the flow of spatial and feature information necessary for accurate object detection. The improvements achieved with the introduction of EfficientNet B3 include a significant boost in computational efficiency, especially in environments with limited computational resources. Additionally, we expect higher levels of accuracy, particularly with high-resolution input images. Compared to typical models that use older or less efficient architectures, EfficientNet B3 offers better scaling and adaptation to modern detection requirements. Moreover, EfficientNet B3 provides significant improvements in training and execution efficiency compared to traditional models, which might require more computational resources for similar levels of performance. This change makes YOLOv5 more competitive and versatile, allowing optimization across various use cases such as real-time image analysis or high-resolution data processing. Overall, incorporating EfficientNet B3 into YOLOv5 represents a substantial upgrade that leverages the latest advancements in CNN techniques to offer a more robust and competitive solution for object detection tasks.

4 Experimental Evaluation

4.1 Evaluation of the Combined Models

For training the YOLOv5 model, we utilized Google Colab with its Tesla T4 GPU, which offers 16 GB of GDDR6 memory and 2,560 CUDA cores. We train the YOLOv5 model for 100 epochs using default parameters. A part of the Stanford dataset⁸, which contains images of different cars, and a truck dataset⁹, specifically including trucks, are combined to create a robust dataset that includes diverse vehicle types, enhancing detection accuracy. For other processes and detections with YOLOv5, we used Visual Studio Code on a system running Windows 11 Home 64-bit, powered by an AMD Ryzen 5 5600G CPU and equipped with 16 GB of DDR4 RAM. The system is based on a Gigabyte B450M K-CF motherboard and features an ATI AMD Radeon Graphics card for graphical processing. Storage is managed by a 1 TB Western Digital WD Blue SN570 SSD, and the display resolution is set to 1920x1080 at 72 Hz. The videos used for object detection were sourced from online platforms and are free for Academic use, ensuring that the data used for testing and training was legally compliant and accessible. ^{10 11 12}

Training time is the duration it takes to train an object detection model, influencing its efficiency and overall performance. Precision (P) measures the accuracy of positive predictions as the ratio of true positives to total predicted positives, while Recall (R) evaluates the model’s effectiveness in identifying relevant instances, calculated as true positives over all actual positives. High precision indicates fewer false positives, whereas high recall reflects effective detection of relevant objects. Additionally, mean Average Precision (mAP) assesses model performance, with mAP@50 focusing on an IoU threshold of 0.5, and mAP@50-95 evaluating across multiple IoU thresholds (0.5 to 0.95). The table presents the performance metrics for various YOLOv5 models trained on two datasets: Coco128 and a custom project dataset. Training times for models using the project dataset are significantly longer, with YOLOv5 taking 65.5 minutes compared to just 12.18 minutes for Coco128.

Precision values were high across both datasets, with YOLOv5 achieving 0.903 on the project dataset, indicating effective identification of true positives. However, recall values dropped to 0.764 for the project dataset, reflecting challenges in detecting all relevant objects, suggesting a need for improved model tuning and dataset quality.

Mean Average Precision (mAP) metrics show that both YOLOv5 models performed well on the Coco128 dataset, achieving an mAP50 of 0.950, while they struggled with the project dataset, obtaining

⁸<https://www.kaggle.com/datasets/jessicali9530/stanford-cars-dataset>

⁹<https://universe.roboflow.com/ilham-winar/venom>

¹⁰<https://www.youtube.com/watch?v=B0YjuKbVZ5w>

¹¹<https://www.youtube.com/watch?v=ByED80IKdIU>

¹²<https://www.vecteezy.com/video/18733751-cars-drive-along-the-embankment-of-the-highway-in-kiev-ukraine-traffic-in-both-directions-on-a-busy-road-aerial-view-of-cityscape-ukraine-kiev-june-1-2020>

only 0.815. The EfficientNet variant showed an improvement in mAP50-95 scores on Coco128 at 0.780, compared to 0.745 for the standard model.

Table 1: Training Results

Model	Train time	P	R	mAP50	mAP50-95
YOLOv5 Coco128	12.18 min	0.891	0.920	0.950	0.745
YOLOv5 Project Dataset	65.5 min	0.903	0.764	0.815	0.633
YOLOv5_eff_net Coco128	14.16 min	0.889	0.923	0.949	0.780
YOLOv5_eff_net Project Dataset	67.08 min	0.904	0.769	0.815	0.654

The following figures illustrate the differences in detection performance between the standard YOLO model and the modified YOLO model using EfficientNet B3. Both images show the same scene however, the YOLO model with EfficientNet B3 exhibits higher confidence scores. This indicates an enhanced ability to detect and classify objects more accurately compared to the standard YOLO. The results highlight the effectiveness of integrating EfficientNet B3 into the YOLO architecture, leading to improved performance in object detection tasks.



Figure 2: Detect with Original YOLOv5



Figure 3: Detect with YOLOv5 EfficientNet

4.2 Car detection in Videos

After successfully training YOLOv5 and acquiring the necessary data, we can utilize the SORT algorithm to enhance detection accuracy. By implementing a code that processes the best results from the YOLOv5 training (specifically the best.pt file) and takes a video as input, we generate a CSV file containing detailed car data for each frame. Each car is identified with a unique name, such as "car1," "car2," etc. The data includes coordinates from the bounding boxes enclosing the cars, the detected class, and the confidence of the detection. The detections and corresponding CSV files are organized by frames, allowing for detailed tracking and analysis of each vehicle's movement and behavior throughout the video.

During detection, we monitor the movement of cars in the video as it plays back normally. This process enables us to concurrently verify the accuracy of our detections, making the detection process

more comprehensive and insightful for data analysis. Additionally, we extract a new video that includes detection overlays and save this video along with a CSV file containing detailed information about the detected objects, including frame numbers, bounding box coordinates, and object IDs. This output provides a complete record of the detections, facilitating further analysis and validation of the detection results.

Figure 4 illustrates the detections made by the model, showcasing the class of each detected object and its corresponding identification number.

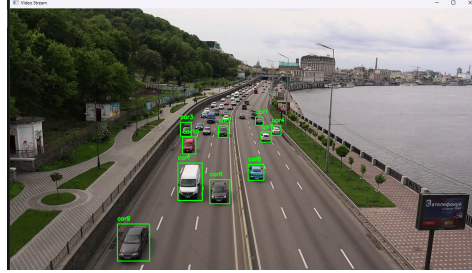


Figure 4: Cars Detection

4.3 Speed Estimation

The process of analyzing the speed of vehicles begins with reading a CSV file containing bounding box coordinates for each detected car, with columns for frame number, object ID, and bounding box coordinates (X_{min} , Y_{min} , X_{max} , Y_{max}). For each car, it calculates the Euclidean distance between the centers of bounding boxes in consecutive frames. The center of a bounding box is computed as the average of the x and y coordinates of the bounding box's corners. Specifically, for bounding boxes $box1$ and $box2$, the distance between their centers is given by:

$$\text{Distance} = \sqrt{\left(\frac{X_{max1} + X_{min1}}{2} - \frac{X_{max2} + X_{min2}}{2}\right)^2 + \left(\frac{Y_{max1} + Y_{min1}}{2} - \frac{Y_{max2} + Y_{min2}}{2}\right)^2}$$

Next, the frame rate (frames per second, FPS) of the video is retrieved using OpenCV. For instance, if the FPS is 30, the time interval between frames is $\frac{1}{30}$ seconds (approximately 0.0333 seconds). The speed for each car is then calculated using the formula:

$$\text{Speed (km/h)} = \frac{\text{Distance (meters)} \times \text{Scale Factor}}{\text{Time Interval (seconds)}} \times 3.6$$

where the scale factor converts pixel distance to meters. For example, if a car moves 50 meters in a time interval of 0.0333 seconds, its speed would be:

$$\text{Speed} = \frac{50 \text{ meters}}{0.0333 \text{ seconds}} \times 3.6 \approx 3,600 \text{ km/h}$$

This formula provides an accurate estimate of the car's speed based on the distances traveled between frames and the frame rate of the video. In the next table 2, we can see the outputs we get for the average speeds of cars in figure 4.

Car	Average Speed (km/h)
Car 5	69.98
Car 6	64.55

Table 2: Average speeds of cars in figure 4

4.4 Detection of Traffic Violations

To evaluate whether a car has or has not stopped at a stop sign, we analyze the video in consecutive frames. A critical factor is the car's immobility in consecutive frames, indicating that the car indeed stopped. In our case, we assume there is a stop sign at a point in the video, as we do not have footage with an actual stop sign and vehicles recorded by a camera that would allow us to analyze our specific requirements. This approach allows us to estimate whether the car complies with the stop sign, although there is no direct visual representation of the sign on the road. If there was signaling through the position of the bounding box, we could also see at which point it exists. Unfortunately, we were unable to find any video featuring a stop sign on a road or with similar others to locate our own. In the next Table 3, we can see the outputs we get for the average speeds of cars and that they did not stop at the stop sign as shown in our video in Figure 4.

Car	Average Speed (km/h)	Status
Car 5	69.98	Did not stop at the stop sign
Car 6	64.55	Did not stop at the stop sign

Table 3: Average speeds and stop sign compliance of cars in figure 4



Figure 5: Stop Sign Detection

4.5 Estimation of Real-Time Traffic Congestion

Using code and the CSV file generated from car detection in a live stream video sourced from a YouTube URL, we can calculate the number of cars present in each frame to assess traffic conditions. For our analysis, we applied this method to two examples: the Grand Avenue Bridge in Glenwood Springs ¹³ and '4 Corners Camera Downtown' ¹⁴. In these cases, we monitored the detected vehicles without a fixed threshold, allowing for flexibility based on varying traffic conditions. By generating a new video with detection overlays, we facilitated real-time monitoring and verification of detected objects. Additionally, we compared the speeds of vehicles observed at the beginning of the monitoring period with those detected later. An increase in the number of vehicles coupled with a decrease in their speeds indicated potential traffic congestion, demonstrating our adaptive approach to effectively managing and analyzing traffic patterns across different road scenarios.

¹³<https://www.youtube.com/watch?v=B0YjuKbVZ5w>

¹⁴<https://www.youtube.com/watch?v=ByED80IKdIU>

In the above picture 6, we see an example of live street camera feed showcasing the real-time detection of vehicles. Each car is identified with a unique label, such as "car13," "car16," and "car20," demonstrating the efficiency of the modified YOLO model in tracking moving objects live.



Figure 6: Grand Avenue Bridge in Glenwood Springs Live Camera

The command line output provides the average speeds of detected vehicles from the live street camera feed, giving insight into traffic conditions. Each vehicle, labeled by a unique identifier, shows specific speeds: for example, car17 averages 54.14 km/h, while car20 is considerably slower at 12.00 km/h. The overall analysis notes that no significant traffic buildup is detected, indicating a smooth flow of traffic despite the varying speeds. In the next tables, Table 4 and Table 5, we can see the outputs of our program as illustrated in Figure 6.

Car	Average Speed (km/h)
Car 15	20.87
Car 16	33.49
Car 17	54.14
Car 18	36.50
Car 19	35.26
Car 20	12.00

Table 4: Average speeds of cars in figure 6

Observation
No significant traffic buildup detected based on current trends.

Table 5: Traffic Buildup Observation in figure 6

In figure 7, we see our second example of the vehicle detection system applied to a different road scenario, showcasing the detection of multiple vehicles, including cars and a truck. The modified YOLO model effectively identifies and labels each vehicle in real time, demonstrating its robustness in various environments.

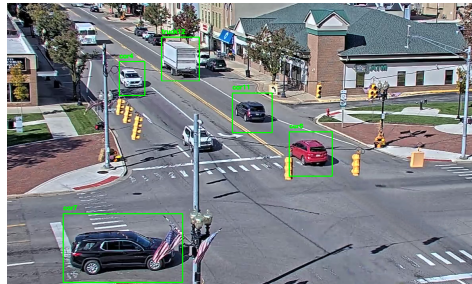


Figure 7: 4 Corners Camera Downtown

Table 6 presents the results derived from the detections. The output details the average speeds of the detected vehicles: car4 travels at an average speed of 14.29 km/h, car11 at 17.87 km/h, and

truck12 at a slower pace of 11.99 km/h. Additionally, Table 7 indicates that no significant traffic buildup was detected based on current trends, suggesting a smooth flow of traffic in this area. This example highlights the model’s effectiveness in monitoring traffic conditions and provides valuable data for traffic management and urban planning efforts.

Vehicle	Average Speed (km/h)
Car 4	14.29
Car 11	17.87
Truck 12	11.99

Table 6: Average speeds of vehicles in figure 7

Observation
No significant traffic buildup detected based on current trends.

Table 7: Traffic Buildup Observation in figure 7

5 Real-World Applications

The potential real-world applications of this system is significant. One key use case is the integration of the system with smart city infrastructure. By connecting it to existing traffic management systems, the system can provide real-time alerts and insights, helping city planners and traffic control centers respond quickly to violations and congested areas. For example, traffic violation alerts could be automatically sent to law enforcement or traffic management teams for immediate action, such as issuing tickets or diverting traffic. Additionally, the system can be integrated into automated law enforcement tools, such as those used by traffic police for detecting speed violations, red light infractions, and illegal parking. By automating these tasks, the system can reduce the workload on human operators and increase the efficiency and consistency of traffic law enforcement. Another potential application is in autonomous vehicles and driver assistance systems. By integrating this real-time detection and monitoring system, autonomous vehicles could better interpret and comply with traffic laws, improving their safety and reliability on the roads. Lastly, the system could play a significant role in enhancing road safety for vulnerable users such as pedestrians and cyclists. With future updates to detect pedestrian crossings, bike lanes, and other safety indicators, the system could contribute to creating safer and more efficient urban environments.

6 Conclusions and Future Directions

Using the YOLO (You Only Look Once) model, we developed an innovative system for car detection, speed estimation and traffic congestion monitoring. Our system records the position of cars in the visual field and calculates their speed based on the change in position between successive frames. Additionally, the system has the capability to recognize stop signs, and upon detecting a stop sign, it monitors the speed of cars, issuing alerts if a car does not come to a complete stop. This aids in enforcing traffic rules and ensuring compliance with signage at intersections. By enhancing the YOLOv5 architecture with EfficientNet B3 as the backbone, we significantly improved performance and accuracy. These enhancements allow the system to operate with greater precision and reliability, which boosts its effectiveness in real-time detection and speed estimation. Looking ahead, several enhancements are planned to expand the system’s capabilities. One key improvement is the expansion of the vehicle detection framework to include various vehicle types, such as buses, motorcycles, bicycles, and heavy-duty trucks. We also plan to enhance the system’s ability to recognize a wider array of road signs, including speed limits, yield signs, and no-entry signs. Integrating new detection models like transformer-based architectures or specialized models for text detection will enable the system to read and interpret road signs more effectively, thereby improving its ability to monitor and enforce compliance with traffic laws. In terms of technical improvements, we aim to enhance the accuracy of

speed estimation by refining the distance measurements and improving the calibration between video pixels and real-world distances.

References

- [1] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *arXiv preprint arXiv:1905.11946*, May 2019.
- [2] Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). "YOLOv4: Optimal Speed and Accuracy of Object Detection." Available at: <https://arxiv.org/abs/2004.10934>.
- [3] Du, W., Dash, A., Li, J., Wei, H., & Wang, G. (2023). "Safety in Traffic Management Systems: A Comprehensive Survey." Ying Wu College of Computing, New Jersey Institute of Technology, Newark, NJ 07102, USA.
- [4] K. Kumar, K. Kumar, and C.L.P. Gupta, "Object Detection in Video Frames using Deep Learning," *International Journal of Computer Applications*, vol. 183, no. 51, pp. 33-39, Feb. 2022.
- [5] Nugroho, A. D., & Baihaqi, W. M. (2023). "Improved YOLOv5 with Backbone Replacement to MobileNet V3s for School Attribute Detection." Universitas Amikom Purwokerto, Indonesia.
- [6] P. Patil. Applications of Deep Learning in Traffic Management: A Review. *International Journal of Business Intelligence and Big Data Analytics*, 5(1), 16–23, 2022.
- [7] V. Pavlov, V. Khryashchev, E. Pavlov, and L. Shmaglit, "Application for Video Analysis Based on Machine Learning and Computer Vision Algorithms," in *Proceedings of the 14th Conference of FRUCT Association*, Yaroslavl, Russia, 2018, pp. 91-100.
- [8] Premaratne, P., Kadhim, I. J., Blacklidge, R., & Lee, M. (2023). "Comprehensive review on vehicle detection, classification and counting on highways." School of Electrical, Computer and Telecommunications Engineering, University of Wollongong, North Wollongong, NSW 2522, Australia; Electrical Engineering Technical College, Middle Technical University, Baghdad, Iraq.
- [9] J. Redmon, S.K. Divvala, R.B. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788, 2015.
- [10] A. Tonge, S. Chandak, R. Khiste, U. Khan and L. A. Bewoor, "Traffic Rules Violation Detection using Deep Learning," 2020 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, 2020.
- [11] Jocher, G., et al. (2023). "YOLOv5, YOLOv8 and YOLOv10: The Go-To Detectors for Real-time Vision." Ultralytics.
- [12] Yu Zhang, Zhongyin Guo, Jianqing Wu, Yuan Tian, Haotian Tang, and Xinming Guo. Real-Time Vehicle Detection Based on Improved YOLO v5.